



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

ตัวดำเนินการ นิพจน์

พศ.ดร.กอบเกียรติ์ สระอุบล
Kobkiat.s@sci.kmutnb.ac.th

อาจารย์เอิร์น สุริยะฉาย
earn.s@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

นิพจน์ (Expression)

- หมายถึง สิ่งี่ประมวลผลแล้วให้ค่าผลลัพธ์ออกมา ซึ่งประกอบด้วย
 - ตัวกระทำการ (Operator)
 - ตัวถูกกระทำการ (Operand)

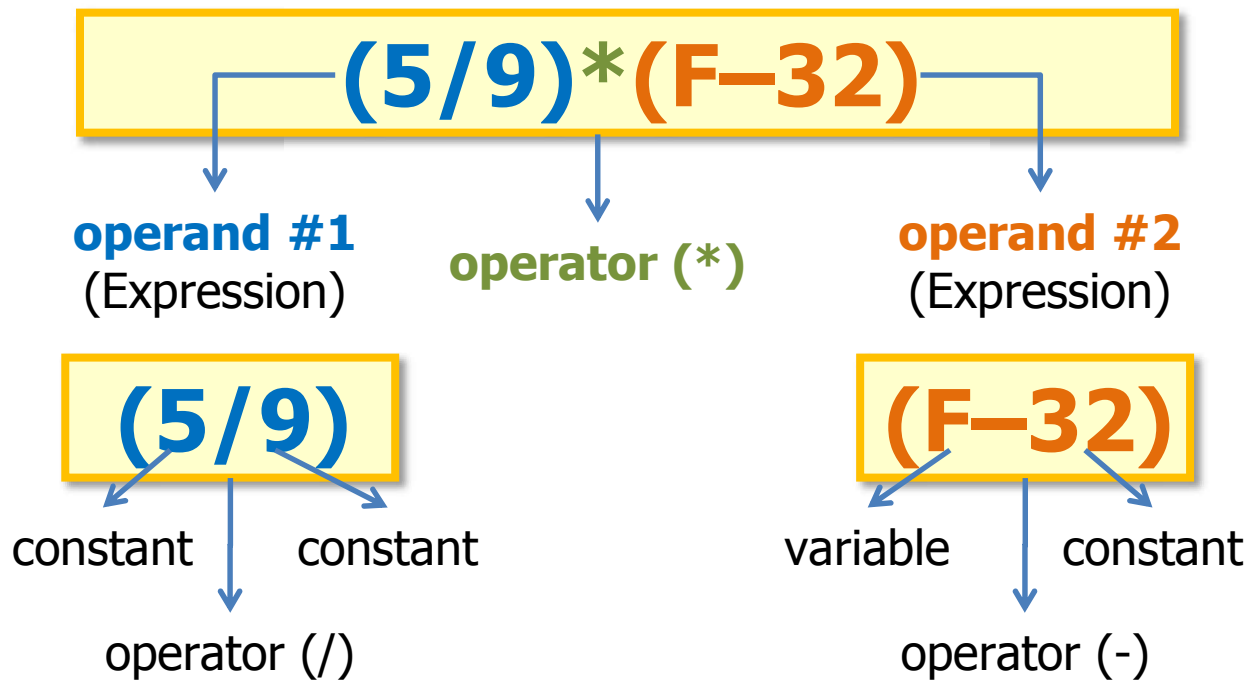
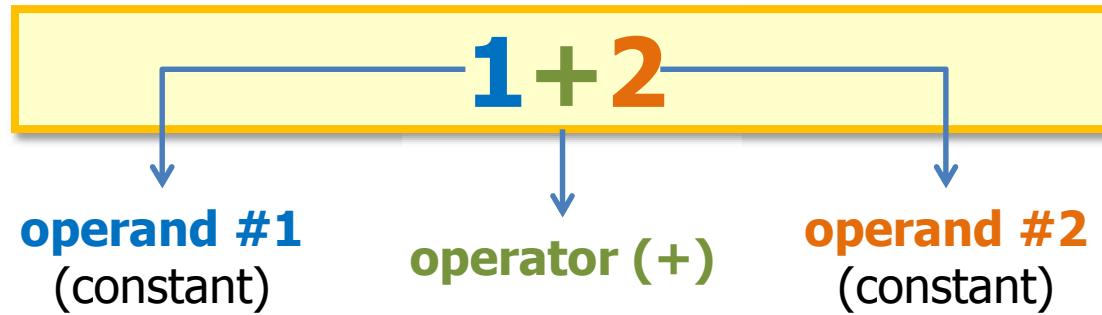
<code>pi = 3.14</code>	<code>total = 100</code>	<code>vat = 0.07</code>	<code>gpa = 2.01</code>
<code>2+3</code>	<code>3*5</code>	<code>8/4</code>	<code>13%5</code>
<code>i++</code>	<code>++i</code>	<code>n--</code>	<code>--n</code>
<code>sizeof(int)</code>	<code>2<5</code>	<code>1 3</code>	<code>(int)a</code>
<code>a += b</code>	<code>a&b</code>	<code>c<<2</code>	<code>(a>b) ? a : b</code>
<code>2*(pi*r)</code>	<code>C = (5/9)*(F-32)</code>		<code>5*a+2*(b-c)</code>

ทุกนิพจน์สามารถหาค่าได้

ตัวดำเนินการ และ ตัวถูกดำเนินการ

- **ตัวดำเนินการ (Operator)** คือ เครื่องหมายหรือสัญลักษณ์ที่ใช้ในการแทนการกระทำอย่างใดอย่างหนึ่งกับข้อมูล
 - เช่น $+$ $-$ $*$ $/$ $\%$ $=$ $>$ $<$ $\&\&$ $\|$
- **ตัวถูกดำเนินการ (Operand)** คือ ข้อมูลที่ถูกกระทำโดยตัวดำเนินการ ซึ่งตัวถูกดำเนินการอาจอยู่ในรูปของ
 - ตัวแปร (Variable)
 - ค่าคงที่ (Constant)
 - ค่าที่ได้รับจากฟังก์ชัน (Return Function)
 - นิพจน์ (Expression)

นิพจน์ (Expression)

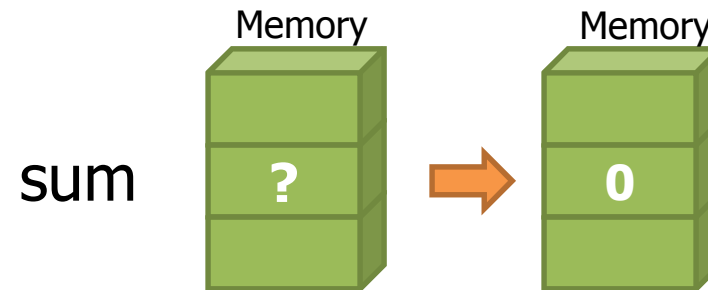


ตัวดำเนินการกำหนดค่า (Assignment Operator)

ตัวแปร = นิพจน์;

- นำผลลัพธ์ของนิพจน์ด้านขวามาเก็บไว้ในตัวแปรด้านซ้าย

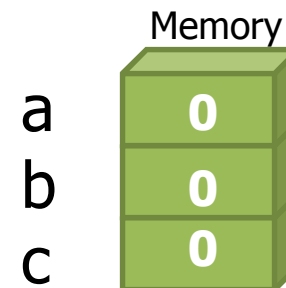
```
int sum;  
sum = 0;
```



- Multiple assignment expression

ตัวแปร1 = ตัวแปร2 = ... = นิพจน์;

```
int a, b, c, C, F;  
a = b = c = 0;  
C = (5/9)*(F-32)
```



ตัวดำเนินการกำหนดค่า (Assignment operator)

- สิ่งที่ต้องระวัง เมื่อใช้ตัวดำเนินการกำหนดค่าแบบหลายค่าพร้อมกัน (Multiple assignment expression) คือ ชนิดของตัวแปรที่ใช้ในเก็บค่าต้องสัมพันธ์กับข้อมูล เช่น

```
int b;  
float a,c;  
a = b = c = 3.14;
```

- จะได้ผลลัพธ์เป็น

```
c = 3.14  
b = 3  
a = 3.00
```



ตัวดำเนินการเลขคณิต (Arithmetic operators)

การคำนวณ	ตัวดำเนินการ	ตัวอย่าง	การทำงาน
บวก	+	$z = x + y;$	นำค่า x บวกกับค่า y แล้วเก็บผลลัพธ์ไว้ใน z
ลบ	-	$z = x - y;$	นำค่า x ลบด้วยค่า y แล้วเก็บผลลัพธ์ไว้ใน z
คูณ	*	$z = x * y;$	นำค่า x คูณกับค่า y แล้วเก็บผลลัพธ์ไว้ใน z
หาร	/	$z = x / y;$	นำค่า x หารด้วยค่า y แล้วเก็บผลลัพธ์ไว้ใน z - ถ้าตัวตั้งและตัวหารเป็นจำนวนเต็มผลลัพธ์จะเป็นจำนวนเต็มด้วย - ถ้าตัวตั้งหรือตัวหารเป็นจำนวนจริงผลลัพธ์ที่ได้จะเป็นจำนวนจริง มีจุดทศนิยมได้
มอดุลัส	%	$z = x \% y;$	นำค่า x หารกับค่า y แล้วเก็บเศษไว้ใน z

ตัวดำเนินการเลขคณิต (Arithmetic operators)

- ในกรณีที่นิพจน์มีตัวดำเนินการหลายตัว จะต้องดำเนินการตามลำดับ
- เช่น $a + b * (c - d) / e$

1. $c - d$

2. $b * (1)$

3. $(2) / e$

4. $a + (3)$

ลำดับ	Operators	การกระทำ
1	()	ย้ายไปขวา
2	* / %	ย้ายไปขวา
3	+ -	ย้ายไปขวา

หมายเหตุ ดูเพิ่มเติมได้ในเรื่อง ลำดับการดำเนินการ

Example # 1

```
#include <stdio.h>
int main()
{
    int a, b, c;
    scanf ("%d", &a);
    scanf ("%d", &b);
    c = a + b;
    printf ("A + B = %d\n", c);
    printf ("A - B = %d\n", a - b);
    printf ("A * B = %d\n", a * b);
    printf ("A / B = %d\n", a / b);
    printf ("A mod B = %d\n", a %
b);
}
```

13
5
A + B = 18
A - B = 8
A * B = 65
A / B = 2
A mod B = 3

10
5
A + B = 15
A - B = 5
A * B = 50
A / B = 2
A mod B = 0

การเขียนนิพจน์ทางคณิตศาสตร์

$$2x^2 + 4x + 3$$



2 * x * x + 4 * x + 3

$$\frac{3x - y}{x}$$



(3 * x - y) / x

$$\frac{3(x - y)}{x - 1} + 2$$



(3 * (x - y) / (x - 1)) + 2

ตัวดำเนินการประกอบ (+= -= *= /= %=)

- ช่วยให้เขียนข้อความสั้นได้สั้น และเร็วขึ้น

ตัวแปร = ตัวแปร **ตัวดำเนินการ** **นิพจน์**;



ตัวแปร **ตัวดำเนินการ** = **นิพจน์**;

การทำงาน	ตัวดำเนินการประกอบ	ลดรูปเป็น
$x = x + 10$	$+=$	$x+=10$
$x = x - 10$	$-=$	$x-=10$
$x = x * y$	$*=$	$x*=y$
$x = x / y$	$/=$	$x/=y$
$x = x \% 3$	$\%=$	$x\%=3$
$x = x + y/3$	$+=$	$x+=y/3$

ตัวดำเนินการเพิ่ม ลดค่า (++ --)

เครื่องหมาย	การคำนวณ	ตัวอย่าง	ความหมาย
++	เพิ่มค่า ทีละหนึ่ง	++a	เพิ่มค่า a ก่อนแล้ว นำ a ไปใช้งาน
		a++	นำค่า a ไปใช้งานก่อน แล้วเพิ่มค่า a
--	ลดค่า ทีละหนึ่ง	--a	ลดค่า a ก่อนแล้ว นำ a ไปใช้งาน
		a--	นำค่า a ไปใช้งานก่อน แล้วลดค่า a

a++ , ++a

```
int a = 10;  
b = ++a;
```



```
a = 10  
a = a+1 = 11  
b = a+1 = 11
```

a = 11 , b = 11

```
int a = 10;  
b = a++;
```



```
a = 10  
b = a = 10  
a = a+1 = 11
```

a = 11 , b = 10

a-- , --a

```
int a = 10;  
b = --a;
```



```
a = 10  
a = a-1 = 9  
b = a-1 = 9
```

a = 9 , b = 9

```
int a = 10;  
b = a--;
```



```
a = 10  
b = a = 10  
a = a-1 = 9
```

a = 9 , b = 10

ตัวดำเนินการเพิ่ม ลดค่า (++ --)

```
int a = 10;  
b = 10 + ++a * 2;
```



```
a = 10  
a = a+1 = 11  
b = 10+a*2
```

a = 11 , b = 32

```
int a = 10;  
b = 10 + a++ * 2;
```



```
a = 10  
b = 10+a*2  
a = a+1 = 11
```

a = 11 , b = 30

Example # 2

```
#include <stdio.h>
int main()
{
    int a = 5;
    printf("a++   = %d\n", a++);
    printf("after a = %d\n", a);
    a = 5;
    printf("++a   = %d\n", ++a);
    printf("after a = %d\n", a);
    a = 5;
    printf("a--   = %d\n", a--);
    printf("after a = %d\n", a);
    a = 5;
    printf("--a   = %d\n", --a);
    printf("after a = %d\n", a);
}
```

```
a++   = 5
after a = 6
++a   = 6
after a = 6
a--   = 5
after a = 4
--a   = 4
after a = 4
```

- a++ เอา a ไปใช้ก่อนเพิ่ม
- ++a เพิ่มค่า a ก่อนไปใช้
- a-- เอา a ไปใช้ก่อนลดค่า
- --a ลดค่า a ก่อนไปใช้

Example # 3

```
#include <stdio.h>
int main()
{
    int x, y;
    x = y = 0;
    printf("x   y\n");
    printf("%d   %d\n", x++, ++y);
    printf("%d   %d\n", x++, ++y);
    printf("%d   %d\n", x++, ++y);
    printf("%d   %d\n", x++, ++y);
    y = x++;
    printf("%d   %d\n", x, y);
    y = ++x;
    printf("%d   %d\n", x, y);
}
```

x	y
0	1
1	2
2	3
3	4
5	4
6	6

ตัวดำเนินการเปรียบเทียบ (Relational operators)

- เป็นการนำข้อมูล 2 ค่ามาเปรียบเทียบกับ ซึ่งผลลัพธ์ที่ได้จากการเปรียบเทียบ คือ
 - ถ้าเป็น**จริง**จะให้ค่า 1
 - ถ้าเป็น**เท็จ**จะให้ค่า 0

Operator	ความหมาย
>	มากกว่า
>=	มากกว่าหรือเท่ากับ
<	น้อยกว่า
<=	น้อยกว่าหรือเท่ากับ
==	เท่ากับ
!=	ไม่เท่ากับ

Example # 4

```
#include <stdio.h>
int main()
{
    int  a, b;      char c;
    a = b = 10;
    c = 'A';
    printf("%d\n", a == b);
    printf("%d\n", a > b);
    printf("%d\n", a >= b);
    printf("%d\n", b < 0);
    printf("%d\n", b <= a-1);
    printf("%d\n", c == 65);
    printf("%d\n", c > 'B');
    printf("%d\n", c > 'B'-2);
}
```

1
0
1
0
0
1
0
1

ตัวดำเนินการเชิงตรรกะ (Logical operators)

- ใช้สำหรับเชื่อมโยงความสัมพันธ์ระหว่างนิพจน์เชิงตรรกะ ซึ่งผลลัพธ์ของการกระทำจะเป็นไปได้ 2 ค่าคือ
 - ถ้าเป็น**จริง**จะให้ค่า 1
 - ถ้าเป็น**เท็จ**จะให้ค่า 0

Operator	ความหมาย
&&	and
	or
!	not

A	B	A && B	A B	!A
True	True	True(1)	True(1)	False(0)
True	False	False(0)	True(1)	False(0)
False	True	False(0)	True(1)	True(1)
False	False	False(0)	False(0)	True(1)

Example # 5

```
#include <stdio.h>
int main()
{
    int  a, b, c ;
    a = b = 1;
    c = 0;
    printf("%d\n", a && b);
    printf("%d\n", a || b);
    printf("%d\n", b && c);
    printf("%d\n", c || 3);
    printf("%d\n", a && -1);
    printf("%d\n", c || 0);
    printf("%d\n", (a && b) || c);
    printf("%d\n", (a || b) && c);
}
```

1
1
0
1
1
0
1
0

Example # 6

นิพจน์	ค่าตรรกะ
$2 > 3$	False (0)
$10 \geq 10$	True (1)
$10 == 10$	True (1)
$2 != 3$	True (1)
$2 \&\& 0$	False (0)
$-2 \parallel 0$	True (1)
$!0$	True (1)
$!!0$	False (0)
$'A' == 65$	True (1)
$(2>3) \parallel !0$	True (1)
$(5<8) \&\& '0'$	True (1)
$!((2!=3)\parallel 0)$	False (0)

ตัวกระทำการบอกขนาด (Size of operator)

- ใช้บอกขนาดของพื้นที่หน่วยความจำที่จะต้องใช้ในการนำเอาข้อมูล ไปเก็บไว้ในหน่วยความจำ
 - ผลลัพธ์ จะมีหน่วยเป็นไบต์ ชนิดเป็น `int`

`sizeof` ค่าคงที่
`sizeof` (ชนิดของข้อมูล)

Example

sizeof(char)	= 1 byte
sizeof(int)	= 4 byte
sizeof i	= 4 byte
sizeof 10	= 4 byte
sizeof(float)	= 4 byte
sizeof f	= 4 byte
sizeof 2.5	= 8 byte
sizeof(double)	= 8 byte
sizeof(sizeof)	= 4 byte

```
#include <stdio.h>
int main()
{
    int i = 10;
    float f = 2.5;
    printf("sizeof(char)           = %d byte\n", sizeof(char));
    printf("sizeof(int)             = %d byte\n", sizeof(int));
    printf("sizeof i                       = %d byte\n", sizeof i);
    printf("sizeof 10                     = %d byte\n", sizeof 10);
    printf("sizeof(float)                 = %d byte\n", sizeof(float));
    printf("sizeof f                     = %d byte\n", sizeof f);
    printf("sizeof 2.5                   = %d byte\n", sizeof 2.5);
    printf("sizeof(double)               = %d byte\n", sizeof(double));
    printf("sizeof(sizeof)               = %d byte\n", sizeof sizeof);
    printf("sizeof(double)               = %d byte\n", sizeof(double));
}
```

หมายเหตุ DevC ขนาดของ int = 4 bytes

ตัวดำเนินการระดับบิต (Bitwise operators)

- ตัวดำเนินการที่กระทำกับตัวถูกกระทำ ในระดับบิตต่อบิต (0,1) คล้ายภาษาในระดับต่ำ ซึ่งทำให้สามารถทำงานได้เร็วขึ้น

Operator	ความหมาย
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
>>	Shift Right
<<	Shift Left

ตัวดำเนินการระดับบิต (Bitwise operators)

A	B	A & B	A B	A ^ B	~A
1	1	1	1	0	0
1	0	0	1	1	0
0	1	0	1	1	1
0	0	0	0	0	1

ตัวดำเนินการระดับบิต (Bitwise operators)

12 0 0 0 0 1 1 0 0

XOR

4 0 0 0 0 0 1 0 0

12 ^ 4 0 0 0 0 1 0 0 0 =8

12 0 0 0 0 1 1 0 0

0 0 0 0 1 1 ~~0 0~~

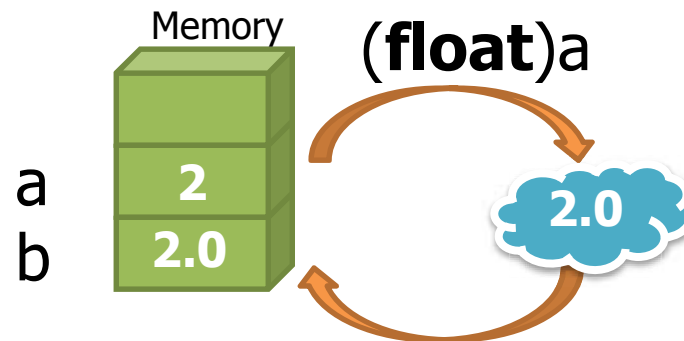
12 >> 2 0 0 0 0 0 0 1 1 =3

ตัวดำเนินการแปลงชนิดข้อมูล (typecast operators)

- ใช้ในการแปลงชนิดของข้อมูลโดยกำหนดชนิดข้อมูลไว้ที่หน้าข้อมูลนั้น

ตัวแปร = (ชนิดข้อมูล)นิพจน์;

```
int a = 2;  
float b;  
b = (float)a;
```



ชนิดข้อมูลของ a ยังเป็น int เหมือนเดิม

Example # 8

```
#include <stdio.h>
int main()
{
    int a, b; a = 2; b = 3;
    float c, f;      f = 4.6;
    c = (float)a;
    printf("%f \n", c);
    c = (float)(a + b);
    printf("%f \n", c);
    a = (int)f + b;
    printf("%d \n", a);
    printf("%f \n", f + b);
}
```

```
2.000000
5.000000
7
7.600000
```

การแปลงชนิดข้อมูล Implicit type conversion

- ถ้าในนิพจน์ประกอบด้วยข้อมูล (ตัวแปร, ค่าคงที่) ที่มีชนิดข้อมูลต่างกัน จะต้องแปลงเป็นชนิดเดียวกันก่อน แล้วจึงดำเนินการ
 - เช่น ถ้าจะหาผลลัพธ์ของ $a + b$, a และ b ต้องเป็นข้อมูลชนิดเดียวกันจึงจะนำมา บวก กันได้
- การแปลงชนิดข้อมูลแบบ Implicit type conversion ภาษาซีจะทำการแปลงให้โดยอัตโนมัติ
 - แปลงจากชนิดข้อมูลที่มีนัยสำคัญ ต่ำกว่า $\rightarrow$ สูงกว่า
 - เช่น `char  $\rightarrow$  int  $\rightarrow$  float  $\rightarrow$  double`

Significance (นัยสำคัญ)

ชนิดข้อมูล	นัยสำคัญ
long double	 สูงสุด ต่ำสุด
double	
float	
unsigned long int	
long int	
unsigned int	
int	
short	
char	

การแปลงชนิดข้อมูล Implicit type conversion

```
char c;      int i;      float f;      double d;
```

```
( c  +  i ) * ( i  -  f ) / ( f  *  d )
```

int

float

double

float

double

double

ตัวดำเนินการคอมมา (Comma Operator)

- เป็นตัวดำเนินการที่เชื่อม นิพจน์ต่าง ๆ เข้าด้วยกัน เพื่อบอกถึงลำดับการทำงาน โดยการทำงานจะเริ่มจากซ้ายไปขวา



นิพจน์1, นิพจน์2, นิพจน์3,... ;

```
#include <stdio.h>
int main()
{
    int a=1, b=2, i;
    i = (a += 2, a + b);
    printf("%d %d %d\n", a,b,i);
    i = a += 2, a + b;
    printf("%d %d %d\n", a,b,i);
}
```

```
3 2 5
5 2 5
```

ลำดับการดำเนินการ

- ในนิพจน์ที่มีตัวดำเนินการหลายตัว จะต้องดำเนินการตามลำดับต่อไปนี้

ลำดับ	Operator	การกระทำ
1	() [] . ->	ย้ายไปขวา
2	++ -- ! ~ (bitwise NOT) &(address) *(dereference) (type) sizeof	ขวาไปซ้าย
3	* / %	ย้ายไปขวา
4	+ -	ย้ายไปขวา
5	>> <<	ย้ายไปขวา
6	> < >= <=	ย้ายไปขวา
7	== !=	ย้ายไปขวา

ลำดับการดำเนินการ

ลำดับ	Operator	การกระทำ
8	& (bitwise AND)	ซ้ายไปขวา
9	^ (bitwise XOR)	ซ้ายไปขวา
10	(bitwise OR)	ซ้ายไปขวา
11	&& (AND)	ซ้ายไปขวา
12	(OR)	ซ้ายไปขวา
13	? :	ขวาไปซ้าย
14	= += -= *= /= %= ^= &= = >>= <<=	ขวาไปซ้าย
15	, (comma operator)	ซ้ายไปขวา

ลำดับการดำเนินการ

$(a + b) + 2 * ++c \parallel (d * 3 + 5)$

- 1) $(a + b)$
- 2) $d * 3$
- 3) $(d * 3 + 5)$
- 4) $++c$
- 5) $2 * ++c$
- 6) $(1) + (5)$
- 7) $(6) \parallel (3)$

โปรแกรมคำนวณค่าวินาทีให้เป็น ชั่วโมง นาที

- เช่น 4,000 วินาที = 1 ชั่วโมง 6 นาที 40 วินาที (1:6:40)

```
#include <stdio.h>
int main()
{
    int sec, min, hour;
    printf("Enter Second :");
    scanf("%d", &sec);
    hour = sec/3600;
    min = (sec-hour*3600)/60;
    sec = sec%60;
    printf("***** Result *****\n");
    printf("Hour    = %d\n",hour);
    printf("Minute = %d\n",min);
    printf("Second = %d\n",sec);
    printf("%d:%d:%d",hour,min,sec);
}
```

```
Enter Second :4000
***** Result *****
Hour    = 1
Minute  = 6
Second  = 40
1:6:40
```

โปรแกรมคำนวณค่าวินาทีให้เป็น ชั่วโมง นาที

```
#include <stdio.h>
int main()
{
    int sec, min, hour;
    printf("Enter Second :");
    scanf("%d", &sec);
    hour = sec/3600;
    min = (sec-hour*3600)/60;
    sec = sec%60;
    printf("***** Result *****\n");
    printf("Hour      = %d\n",hour);
    printf("Minute = %d\n",min);
    printf("Second = %d\n",sec);
    printf("%d:%d:%d",hour,min,sec);
}
```

เรียกใช้ Library stdio.h

โปรแกรมหลัก main

{ เริ่มต้นโปรแกรมหลัก main

ประกาศตัวแปร sec, min, hour

แสดงข้อความ Enter Second :

รับข้อมูลจำนวนเต็มเก็บใน sec

คำนวณ จำนวนชั่วโมง

คำนวณ จำนวนนาทีที่เหลือ

คำนวณ จำนวนวินาทีที่เหลือ

แสดงข้อความ ***** Result *****

แสดงข้อความและจำนวนชั่วโมง

แสดงข้อความและจำนวนนาที

แสดงข้อความและจำนวนวินาที

แสดงข้อความแบบสรุป

} จบโปรแกรมหลัก main